

INTRODUCCIÓN A GIT Y DOCKER

Javier Becerra Elcinto

javier@auva.es

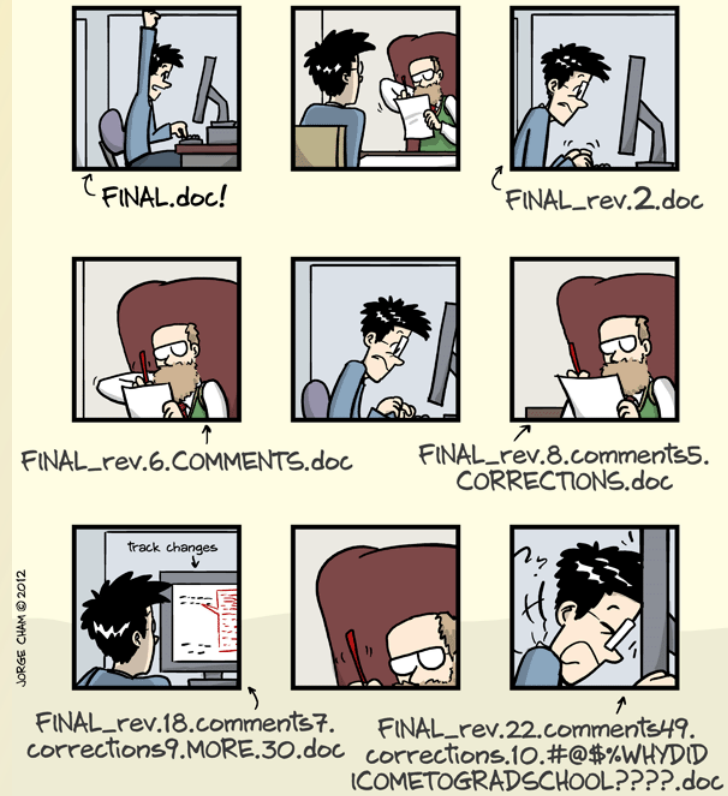
javier@panoimagen.com



CRIE - Noviembre 2023

INTRODUCCIÓN

"FINAL".doc



WWW.PHDCOMICS.COM

AUTOR

Estas transparencias han evolucionado a lo largo de los años. Su punto de partida es el curso *Curso Git Github Markdown 2015* de Adolfo Sanz De Diego twitter.com/asanzdiego y han sido revisadas gracias a los apuntes del curso de **César Brera**.

Licencia: [Creative Commons Reconocimiento-CompartirIgual 3.0](https://creativecommons.org/licenses/by-sa/3.0/)

Las slides originales de Adolfo Sanz de Diego y sus fuentes las podéis encontrar en: <https://github.com/asanzdiego/curso-git-github-markdown-2015>

Esta versión se puede obtener enviando un correo a javier@auva.es

OBJETIVOS

1. Conocer las **características de Git** y ser capaz de instalarlo y configurarlo.
2. Conocer y ser capaz de usar los **comandos de Git** en repositorios locales.
3. Conectarnos a **repositorios remotos**.
4. Gestionar nuestro trabajo con un **flujo de trabajo** adecuado.
5. Conocer y ser capaz de **usar las principales características de GitHub**.

ENLACES A DOCUMENTACIÓN OFICIAL

- Página oficial de Git:
 - <https://git-scm.com/>
- Pro GIT (sobre todo temas 1, 2, 3 y 6):
 - <https://git-scm.com/book/es/v2>
- Página oficial de GitHub:
 - <https://github.com/>

OTROS ENLACES DE INTERÉS

- Aprender GIT... y de paso GitHub:
 - <https://github.com/oslugr/curso-git>
- Minitutorial de GIT:
 - <https://try.github.io/>
- Tutorial de GIT de codecademy;
 - <https://www.codecademy.com/learn/learn-git>
- How GitHub Uses GitHub to Build GitHub:
 - <http://zachholman.com/talk/how-github-uses-github-to-build-github/>

USO BÁSICO DE GIT

SISTEMA CONTROL DE VERSIONES

Sistema que registra los cambios realizados sobre un archivo o conjunto de archivos a lo largo del tiempo, de modo que puedas recuperar versiones específicas más adelante.

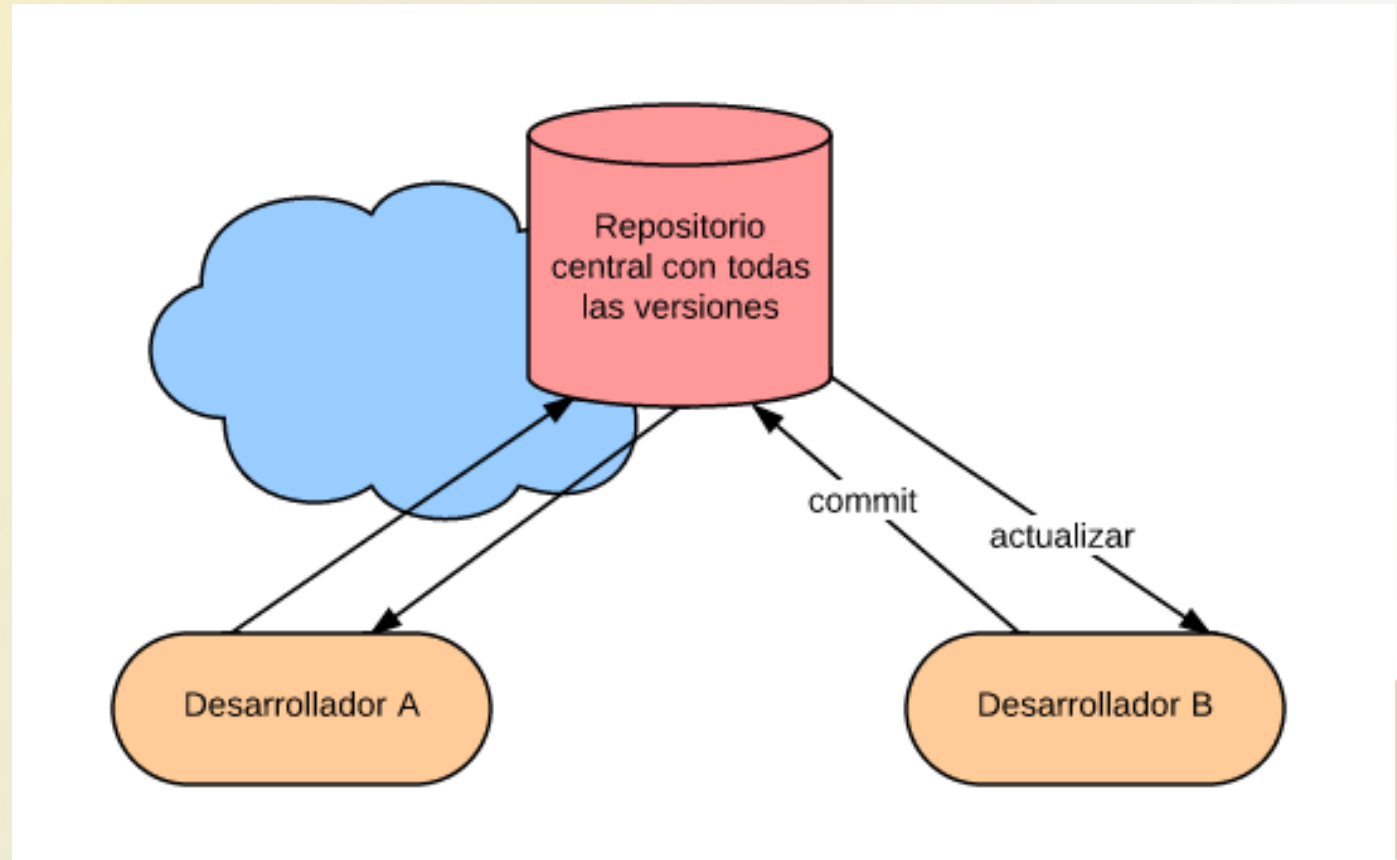
<https://git-scm.com/book/es/v2/Empezando-Acerca-del-control-de-versiones>

VCS LOCALES: UN POCO DE HISTORIA

(VCS => Version Control System)

- **Lo más simple:** hacer copias de directorios.
- Posteriormente aparecieron **bases de datos en local** que guardan el registro de los cambios realizados a los archivos.

VCS CENTRALIZADOS



- Un **servidor central** que guarda los cambios.

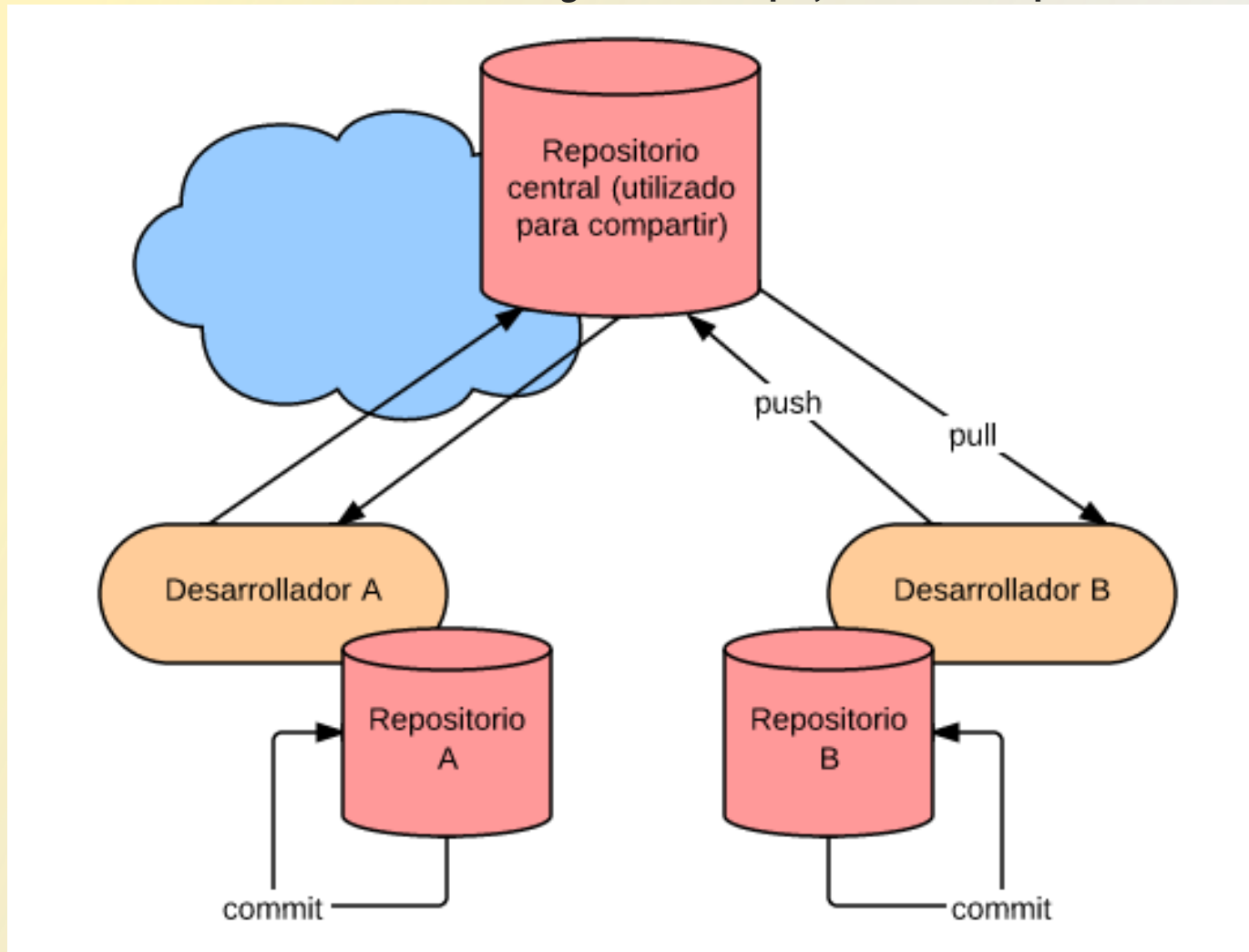
Por ejemplo *Subversion*, *Perforce*, *VisualSourceSafe*...

PROS Y CONTRAS VCS CENTRALIZADOS

- **Pros:** más colaborativo que el local.
- **Contras:** dependes de un servidor central.

VCS DISTRIBUIDOS

- Cada cliente **no solo descarga la última copia, sino todo el repositorio.**



Por ejemplo, *git*, *mercurial*, *fossil*, *bazaar*...

VENTAJAS VCS DISTRIBUIDOS

- Puedes seguir trabajando aunque el repositorio remoto esté caído.
 - **más autonomía**
- La información está más replicada.
 - **menos vulnerable**
- Permite pruebas en local y subir solo lo relevante.
 - **más limpieza**

El más popular de todos ellos es sin duda **GIT**, aunque los conceptos que vamos a aprender son en su mayoría aplicables a cualquier otro sistema VCS distribuido.

GIT

CARACTERÍSTICAS DE GIT

- Creado por **Linus Torvalds**, creador y mantenedor del kernel Linux, en 2005.
- Objetivos cuando se creó:
 - **Rápido**
 - **Sencillo**
 - **Multi rama**
 - **Distribuido**
 - **Grandes proyectos**

INSTALACIÓN

- Windows: <https://git-scm.com/download/win>
- Mac: <https://git-scm.com/download/mac>
- Linux: <https://git-scm.com/download/linux>

CONFIGURACIÓN INICIAL

Antes de hacer el primer *commit* (algo así como una instantánea del estado de nuestro código que queda fijado en la historia de nuestro proyecto) tenemos que identificarnos:

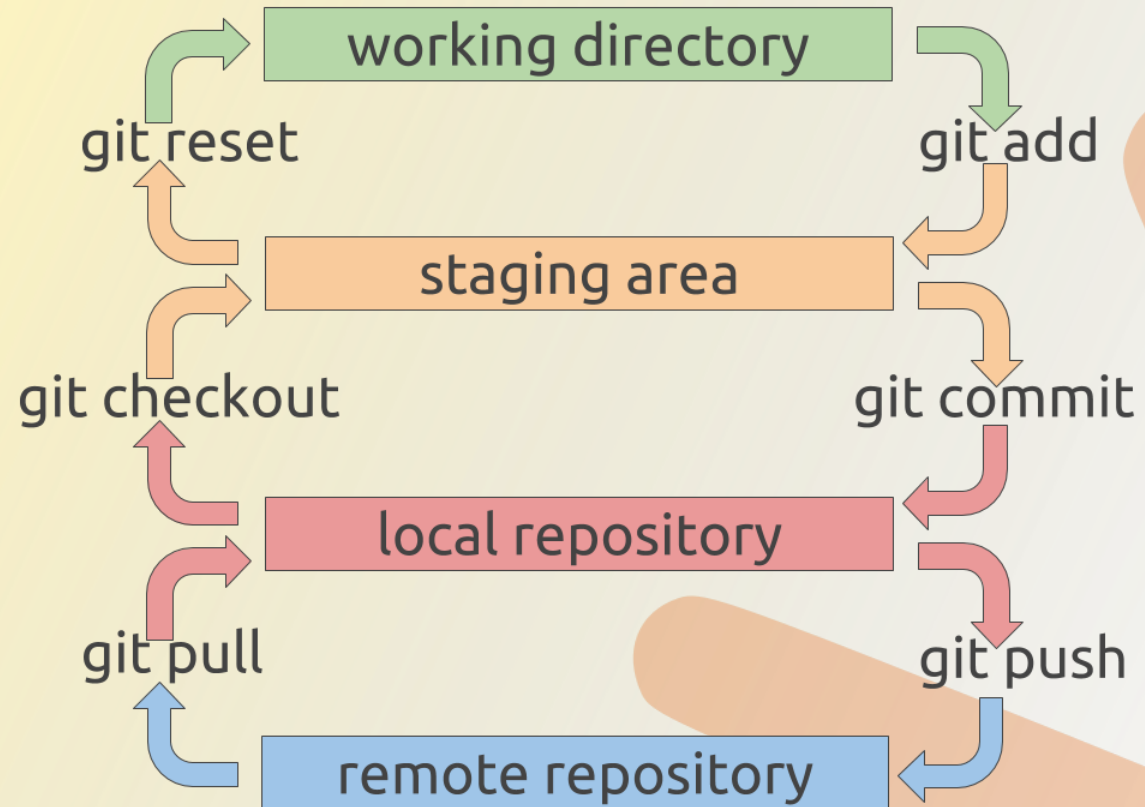
```
git config --global user.name "Nombre Apellidos"  
git config --global user.email "correo@electronico.es"
```

Cada commit que hagamos a partir de ahora estará 'firmado' con nuestros datos.

GUIs

- <https://git-scm.com/downloads/guis>
- La mayoría de *IDEs* (VisualStudio, VisualStudio Code, JetBrains...), incorpora integración con varios sistemas de control de versiones, incluido *Git*.

¿CÓMO FUNCIONA git?



INICIALIZAR UN REPOSITORIO LOCAL

```
git init .
```

- Veremos que se ha creado el subdirectorio `.git` en la carpeta actual con archivos internos de git para gestionar el repositorio.

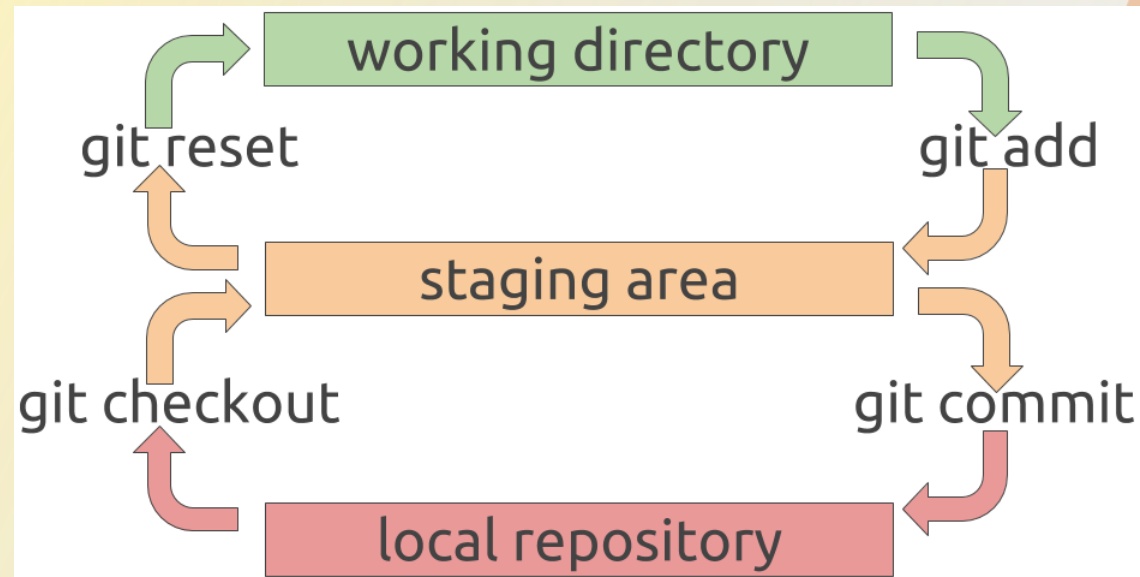
Una vez inicializado, podemos empezar a añadir ficheros al repositorio.

Al inicializar un repositorio:

- Rastreamos todos los cambios en la carpeta raíz (dónde se ha generado la subcarpeta .git) y sus subcarpetas.
- Git rastrea cambios en **ficheros**. Si creamos una subcarpeta vacía **no 'aparecerá' en git**.
Si le añadimos un fichero a esa subcarpeta, ese fichero sí que será visible para git que gestionará la subcarpeta según lo necesite.

EL ÁREA DE STAGING

La zona de 'staging' es una zona intermedia que nos permite marcar algunos cambios para su almacenamiento en el repositorio.



VER EL ESTADO DE LOS ARCHIVOS

- Importante saber el **estado** de nuestro directorio de trabajo frente al área de staging y repositorio local.

```
git status
```

AÑADIR ARCHIVOS

- Podemos **añadir** los **cambios** de un fichero (o varios) al área de staging (desde el área de trabajo).

```
git add nombre-del-fichero  
git add *.extension
```

IMPORTANTE

El área de staging almacena **cambios** los cambios en el fichero en el momento de llamar a `git add`.

La siguiente secuencia:

1. Modificamos el fichero `fichero.txt` poniendo los títulos en mayúsculas.
2. Registramos *los cambios* que acabamos de hacer en el área de staging (`git add fichero.txt`)
3. Modificamos el fichero `fichero.txt` poniendo los títulos en minúsculas.

...en el área de staging estarán registrados los cambios del punto **1** de `fichero.txt` (con los títulos en mayúsculas).

Los cambios que hemos hecho en el punto 3 solo existen en nuestro **directorio de trabajo**.

BORRAR ARCHIVOS

- Podemos **borrar archivos** del área de staging (también lo borrará del área de trabajo)

```
git rm nombre-del-fichero
```

MOVER/RENOMBRAR ARCHIVOS

- Podemos **mover/renombrar archivos** en el área de staging (también lo hará en el área de trabajo)

```
git mv nombre-antiguo nombre-nuevo
```

Nota: si cambiamos el nombre de un fichero desde el sistema operativo (sin utilizar `git mv`), es bastante probable que `git` nos indique que hay dos cambios:

1. Se ha borrado un fichero (con el nombre antiguo)
2. Se ha creado uno nuevo (con el nombre nuevo)

Si no hay muchas diferencias entre la versión antigua y la nueva, `git` combinará esas dos operaciones en una única de 'renombrado' cuando hagamos un *commit*.

RESETEAR ARCHIVOS

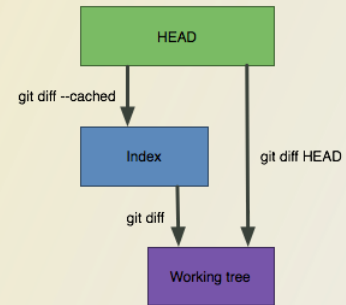
- Para **resetear** los cambios de un fichero (o varios) al área de trabajo (desde el área de staging).

```
git reset nombre-del-fichero
```

VER LAS DIFERENCIAS

- Podemos ver las **diferencias** entre el área de staging y el área de trabajo.

```
git diff
```



GRABAR LOS CAMBIOS

- Para **grabar** los cambios realizados en el repositorio (desde el área de staging):

```
git commit -m "mensaje corto descriptivo"
```

A partir de ahora, la historia de nuestro proyecto incluirá una "instántanea" con el estado del proyecto en ese momento (el estado del *commit* anterior más lo cambios registrados en el área de *staging* en ese momento).

GRABAR LOS CAMBIOS

	COMMENT	DATE
○	CREATED MAIN LOOP & TIMING CONTROL	14 HOURS AGO
○	ENABLED CONFIG FILE PARSING	9 HOURS AGO
○	MISC BUGFIXES	5 HOURS AGO
○	CODE ADDITIONS/EDITS	4 HOURS AGO
○	MORE CODE	4 HOURS AGO
○	HERE HAVE CODE	4 HOURS AGO
○	AAAAAAAAA	3 HOURS AGO
○	ADKFJSLKDFJSDKLFJ	3 HOURS AGO
○	MY HANDS ARE TYPING WORDS	2 HOURS AGO
○	HAAAAAAAAAANDS	2 HOURS AGO

AS A PROJECT DRAGS ON, MY GIT COMMIT MESSAGES GET LESS AND LESS INFORMATIVE.

GRABAR LOS CAMBIOS

- Si queremos utilizar una descripción más completa.

```
git commit
```

- Se abrirá un editor de texto en el que incluiremos:
 1. Primera línea: como descripción sencilla (máximo recomendado 50 caracteres)
 2. Segunda línea: en blanco.
 3. A continuación: descripción detallada de los cambios con la extensión que necesitemos.

¿Y si no puedo describir en una única línea los cambios que he hecho en un commit?

Normalmente es una indicación de que tu *commit* es demasiado complejo, y tendrías que haber realizado commits con conjuntos de cambios más pequeños y centrados en funcionalidad más específica.

GRABAR LOS CAMBIOS... CON VI

 VI Editor

GRABAR LOS CAMBIOS... CON VI

VI/VIM es un editor de texto muy potente y relativamente popular en linux... pero muy poco amigable para el usuario. Si `git` está configurado para utilizarlo en los mensajes de *commit*, aquí tienes las instrucciones para poder cerrarlo y guardar tus cambios:

 Cómo salir de VI

*Resumen: **ESC :wq ENTER** graba los cambios y sale. **ESC :q! ENTER** sale sin grabar los cambios.*

DESHACER LOS CAMBIOS

- Para **deshacer** los cambios de un fichero (o varios) al área de staging (desde el repositorio).

```
git checkout nombre-del-fichero
```

LISTADO DE CAMBIOS

- Para ver el **histórico de cambios** realizados en el repositorio.

```
git log
```

ALIAS

- Podemos crear **alias**:

```
git config --global alias.list 'log --oneline --decorate --graph --all'  
git list
```

IGNORAR ARCHIVOS

- Podemos ignorar archivos añadiéndolos al fichero **.gitignore**.
- Generación automática de archivos *.gitignore* en función del tipo de proyecto: <https://gitignore.io>

CREANDO ETIQUETAS

- Existen etiquetas **ligeras**, y etiquetas **anotadas** (iguales pero estas con más información)

```
git tag nombre-etiqueta-ligera  
git tag -a nombre-etiqueta-anotada -m "mensaje"
```

ETIQUETAS TARDÍAS

- Se puede crear una etiqueta **conociendo el hash del commit** (verlo con git log).

```
git tag -a nombre-etiqueta-annotada -m "mensaje" hash-del-commit
```

VER UNA ETIQUETA

- Podemos **ver información concreta de una etiqueta.**

```
git show nombre-etiqueta
```

SACAR UNA ETIQUETA

- No podemos sacar una etiqueta, pero podemos **colocar en nuestro directorio de trabajo una versión que coincida con alguna etiqueta, creando una rama nueva:**

```
git checkout -b nombre-rama nombre-etiqueta
```

WORKFLOW: A GIT WORKFLOW

UN FLUJO DE TRABAJO CON GIT

FLUJO DE TRABAJO

- Ya conocemos la herramienta de control de versiones.

¿Y cómo la utilizamos en nuestro día a día?

FLUJO DE TRABAJO

 A successful Git branching model

VERSIONADO SEMÁNTICO

VERSIONADO SEMÁNTICO

Utilizando un número de versión con formato MAYOR.MENOR.PARCHES, incrementamos:

1. MAYOR: al introducir cambios que rompen la compatibilidad de la API.
2. MENOR: nueva funcionalidad en la API, compatible con la anterior.
3. PARCHES: solución de *bugs* compatible con la API anterior.

VERSIONADO SEMÁNTICO

<https://semver.org/lang/es/>

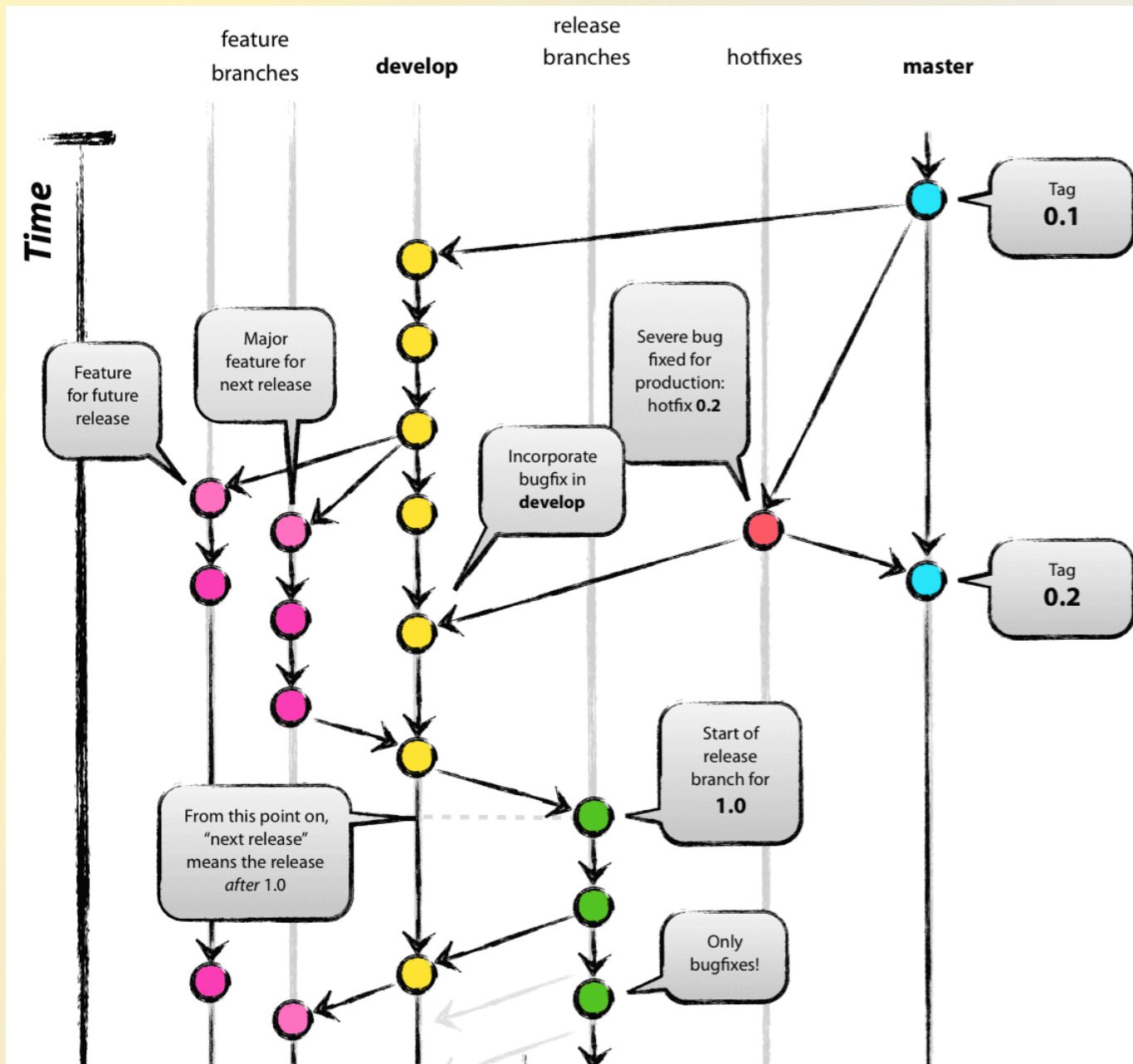
GITHUB

SECCIÓN: USO BÁSICO DE GITHUB

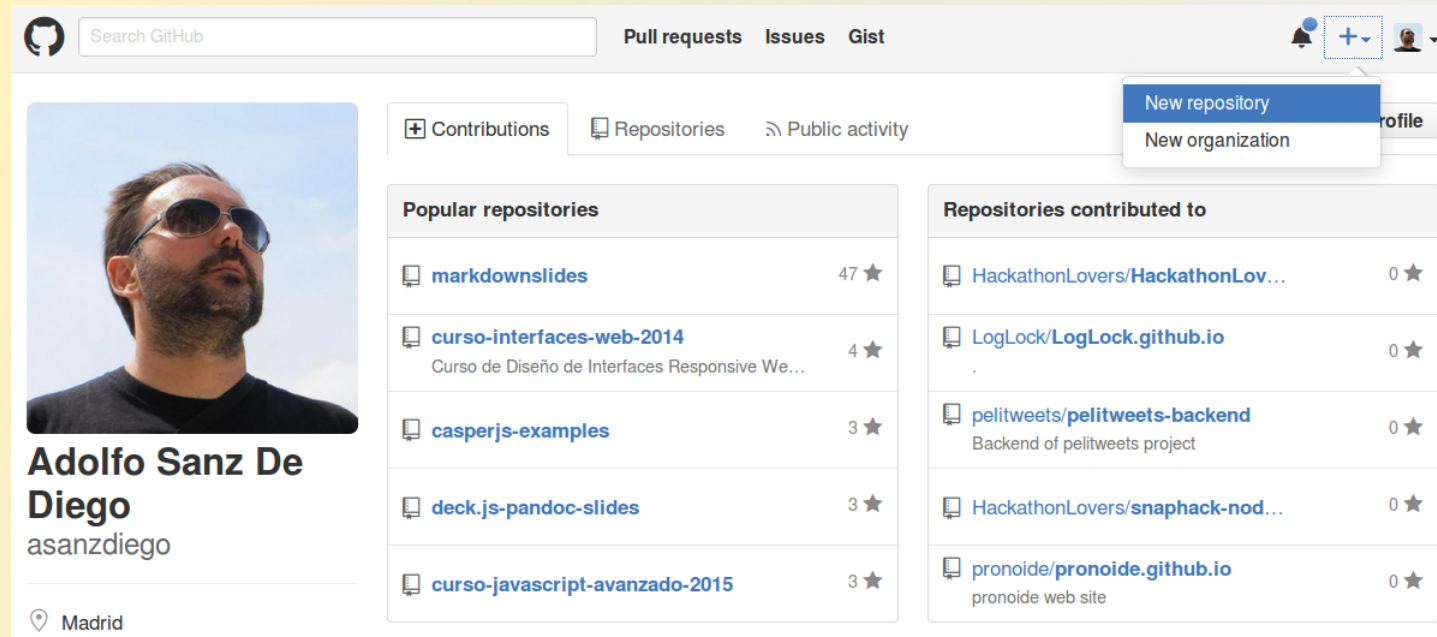
CARACTERÍSTICAS DE GITHUB

- **Plataforma de desarrollo colaborativo**, que utiliza Git.
- Podemos tener **repositorios públicos** o **privados**.
- Tiene facetas de **red social** (perfil público, seguidores, estrellas, etc.).
- Nos permite **gestionar organizaciones y equipos**.
- **Gestión de proyectos** (wiki, releases, incidencias, gráficos, etc.).
- **Servidor web** para páginas sencillas (*GitHub pages*).
- **Integración continua/Despliegue continuo** mediante acciones.

CREAR CUENTA GRATUITA



CREAR REPOSITORIO



Search GitHub

Pull requests Issues Gist

+ profile

Contributions Repositories Public activity

Popular repositories

markdownslides	47 ★
curso-interfaces-web-2014 Curso de Diseño de Interfaces Responsive We...	4 ★
casperjs-examples	3 ★
deck.js-pandoc-slides	3 ★
curso-javascript-avanzado-2015	3 ★

Repositories contributed to

HackathonLovers/HackathonLov...	0 ★
LogLock/LogLock.github.io	0 ★
pelitweets/pelitweets-backend Backend of pelitweets project	0 ★
HackathonLovers/snaphack-nod...	0 ★
pronoide/pronoide.github.io pronoide web site	0 ★

Adolfo Sanz De Diego
asanzdiego

Madrid

New repository
New organization

CONFIGURAR CLAVES (I)

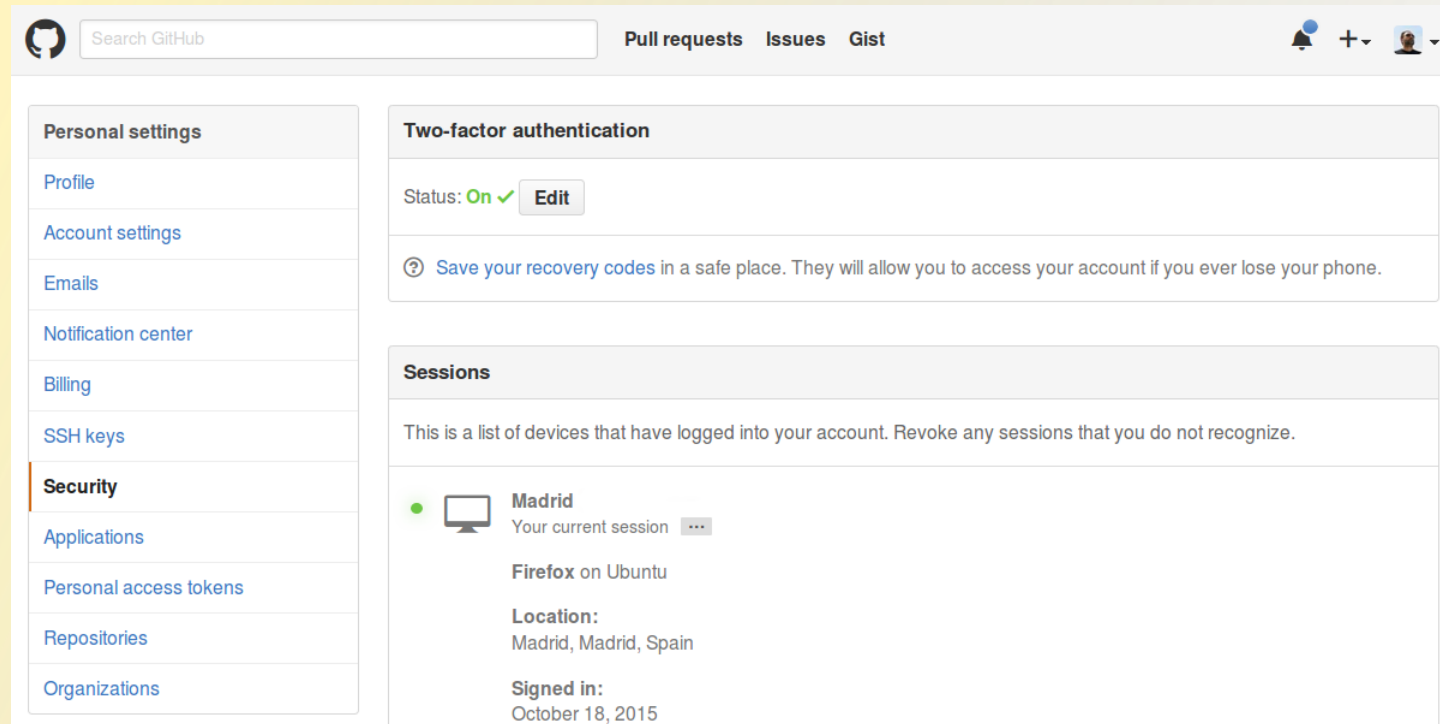
- Nos permite gestionar repositorios **mediante SSH** sin tener que estar poniendo siempre nuestra contraseña.
- Se genera una **clave privada** que se guarde en nuestro ordenador y una **clave pública** que es la que tenemos que guardar en nuestra cuenta.

CONFIGURAR CLAVES (II)

- La podemos usar pues **solo con un ordenador**.
- Instrucciones: <https://help.github.com/articles/generating-ssh-keys/>

DOBLE FACTOR DE AUTENTIFICACIÓN

- View profile and more > Settings > Security



The screenshot shows the GitHub account settings page. On the left is a sidebar with a 'Personal settings' section containing links for Profile, Account settings, Emails, Notification center, Billing, SSH keys, Security (highlighted with a red bar), Applications, Personal access tokens, Repositories, and Organizations. The main content area has two sections: 'Two-factor authentication' and 'Sessions'. The 'Two-factor authentication' section shows a status of 'On' with a green checkmark and an 'Edit' button. Below it is a note about saving recovery codes. The 'Sessions' section shows a list of devices, with one active session from Madrid, Spain, using Firefox on Ubuntu, signed in on October 18, 2015.

Personal settings

- Profile
- Account settings
- Emails
- Notification center
- Billing
- SSH keys
- Security**
- Applications
- Personal access tokens
- Repositories
- Organizations

Two-factor authentication

Status: **On** ✓ [Edit](#)

🔔 [Save your recovery codes](#) in a safe place. They will allow you to access your account if you ever lose your phone.

Sessions

This is a list of devices that have logged into your account. Revoke any sessions that you do not recognize.

-  **Madrid**
Your current session [...](#)
Firefox on Ubuntu
Location:
Madrid, Madrid, Spain
Signed in:
October 18, 2015

USO SOCIAL

- Características sociales:
 - Seguir a gente.
 - Seguir proyectos (watch).
 - Premiar proyectos (start).
 - Forquear proyectos (fork).
 - Crear organizaciones.

SECCIÓN: USO AVANZADO DE GIT

CONECTAR UN REPOSITORIO REMOTO

- Podemos **conectar uno o varios repositorios remotos** a nuestro repositorio.
- (Un *repositorio remoto* también puede ser otra carpeta en nuestro equipo).

```
git remote add alias-repositorio-remoto url-repositorio-remoto
```

RENOMBRAR UN REPOSITORIO REMOTO

- Podemos **renombrar el alias de un repositorio remoto**.

```
git remote rename antiguo-alias nuevo-alias
```

DESCONECTAR UN REPOSITORIO REMOTO

- Podemos **desconectar un repositorio remoto**.

```
git remote remove alias-repositorio-remoto
```

VER LOS REPOSITORIOS REMOTOS

- Podemos **ver los repositorios remotos conectados y los permisos que tenemos.**

```
git remote -v
```

DESCARGAR CAMBIOS REMOTOS

- Podemos **descargar un listado de los cambios disponibles en el repositorio remoto (sin modificar nuestro repositorio local).**

```
git fetch alias-repositorio-remoto
```

DESCARGAR Y COMBINAR

- Podemos **descargar y combinar los cambios remotos** con los de tu repositorio local.

```
git pull alias-repositorio-remoto nombre-rama-repositorio-remoto
```

ENVIAR DATOS (I)

- Podemos **enviar datos al repositorio remoto** (solo si estamos up-to-date).

```
git push alias-repositorio-remoto nombre-rama-repositorio-remoto
```

ENVIAR DATOS (II)

- Normalmente:

```
git push origin master
```

ENVIAR DATOS (III)

- Si queremos subir los tags:

```
git push --tag origin master
```

CLONAR REPOSITORIOS

- Clonar es como:
 - hacer un init
 - luego un remote add
 - luego un fetch con alias=origin
 - dejando las ramas remota y local en master

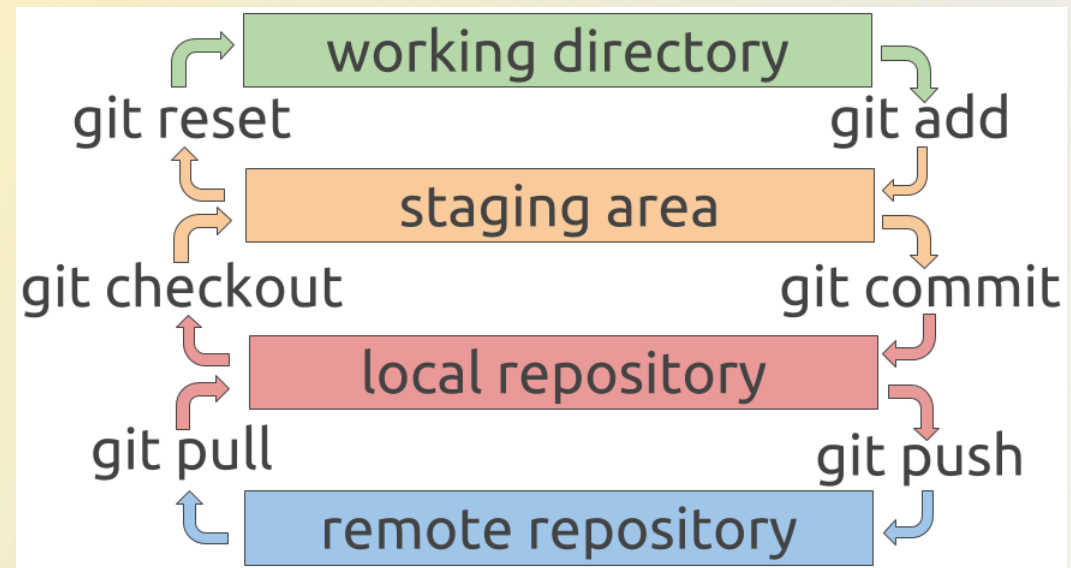
```
git clone url-repositorio-remoto
```

INSPECCIONAR REPOSITORIO REMOTO

- Podemos ver **información de un remoto particular, y como están configurados pull y push.**

```
git remote show alias-repositorio-remoto
```

RESUMEN ÁREAS



CREAR UNA RAMA

- Podemos crear ramas que son **apuntadores que podemos mover por los distintos snapshots**.
- Solo la creamos, no nos situamos en ella.

```
git branch nombre-rama
```

CAMBIAR DE RAMA

- El HEAD es el apuntador que usa GIT para saber en que rama estás.
- Cuando cambiamos de rama GIT **cambia el HEAD y los ficheros de tu área de trabajo.**

```
git checkout nombre-rama
```

CREAR Y CAMBIAR DE RAMA

- Podemos **crear y cambiar de rama** con un mismo comando.

```
git checkout -b nombre-rama
```

VER LAS RAMAS Y EL HEAD

- Podemos **ver las ramas y donde apunta el HEAD**.

```
git log --oneline --decorate --graph --all  
git branch -v
```

FUSIONAR RAMAS

- GIT es **muy potente** con la fusión de ramas.

```
git merge nombre-rama
```

SOLUCIONAR CONFLICTOS

- Si al hacer un merge existen conflictos **GIT los apunta en los propios ficheros.**

```
<<<<<< HEAD:index.html
<div id="footer">contact : email.support@github.com</div>
=====
<div id="footer">please contact us at support@github.com</div>
>>>>>> issue:index.html
```

BORRAR RAMAS

- Una vez fusionado la rama en el master, **conviene borrarla** (solo nos deja si está fusionada).

```
git branch -d nombre-rama
```

LISTADO DE RAMAS POR ESTADO

- Podemos saber **que ramas están fusionada y cuales no.**

```
git branch --merged  
git branch --no-merged
```

SINCRONIZAR RAMA REMOTA

- Igual que sincronizamos la rama master remota, podemos **sincronizar otras ramas remotas**.

```
git checkout -b nombre-rama-local alias-repositorio-remoto/nombre-rama-remota  
git checkout --track alias-repositorio-remoto/nombre-rama-remota
```

ASIGNAR RAMA REMOTA

- Podemos **asignar el área de trabajo a una rama remota**.

```
git checkout -u alias-repositorio-remoto/nombre-rama-remota
```

LISTADO DE TODAS LAS RAMAS

- Podemos listar no solo las ramas locales, sino **también las remotas**.

```
git branch -vv
```

ELIMINAR RAMA REMOTA

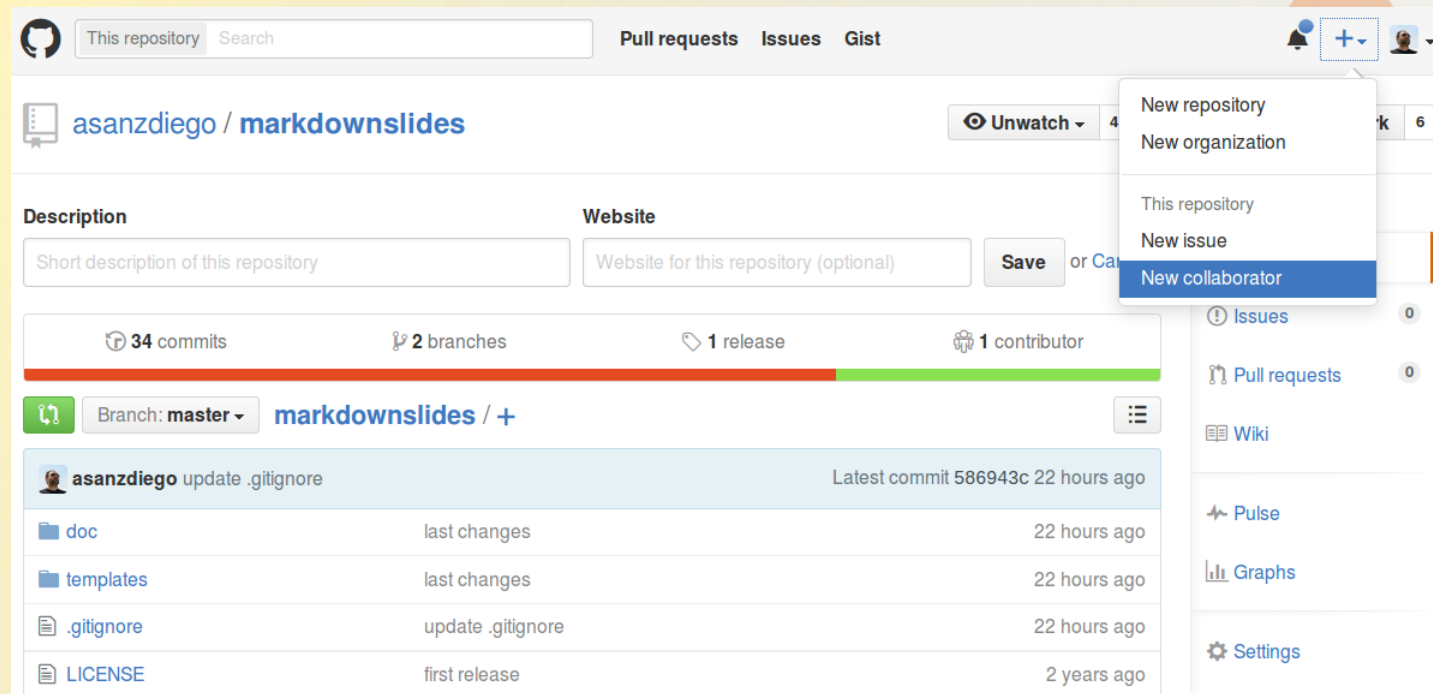
- Podemos **eliminar las ramas remotas**.

```
git push alias-repositorio-remoto --delete nombre-rama-remota
```

SECCIÓN: USO AVANZADO DE GITHUB

AÑADIR COLABORADORES

- Podemos **dar permisos de push** a quien queramos.



The screenshot shows the GitHub repository page for 'asanzdiego/markdownslides'. The repository has 34 commits, 2 branches, 1 release, and 1 contributor. The 'Add new collaborator' dropdown menu is open, showing options: 'New repository', 'New organization', 'This repository', 'New issue', and 'New collaborator' (highlighted). The repository description is 'Short description of this repository' and the website is 'Website for this repository (optional)'. The repository is on the 'master' branch. The commit history shows the latest commit 'asanzdiego update .gitignore' 22 hours ago. The file list includes 'doc', 'templates', '.gitignore', and 'LICENSE'.

File	Commit	Time
doc	last changes	22 hours ago
templates	last changes	22 hours ago
.gitignore	update .gitignore	22 hours ago
LICENSE	first release	2 years ago

CREAR ORGANIZACIONES

- También podemos **crear organizaciones** para gestionar proyectos con varios colaboradores.

 GitHub New Organization